

2026년도 전국대회 과제

직 종 명	클라우드 컴퓨팅	과 제 명	Small Challenge	과제번호	제 2과제
경기시간	4시간	비 번 호		심사위원 확 인	(인)

1. 요구사항

- 1) CDN
- 2) Keycloak
- 3) Container logging
- 4) Workflow

2. 선수 유의사항

※ 다음 유의사항을 고려하여 요구사항을 완성하시오.

- 1) 기계 및 공구 등의 사용 시 안전에 유의하시고, 필요 시 안전장비 및 복장 등을 착용하여 사고를 예방하여 주시기 바랍니다.
- 2) 작업 중 화상, 감전, 찰과상 등 안전사고 예방에 유의하시고, 공구나 작업도구 사용 시 안전 보호구 착용 등 안전수칙을 준수하시기 바랍니다.
- 3) 작업 중 공구의 사용에 주의하고, 안전수칙을 준수하여 사고를 예방하여 주시기 바랍니다.
- 4) 경기 시작 전 가벼운 스트레칭 등으로 긴장을 풀어주시고, 작업도구의 사용 시 안전에 주의하십시오.
- 5) 문제에 제시된 괄호 < >는 변수를 뜻함으로 선수가 적절히 변경하여 사용해야 합니다.
- 6) Security Group의 80/443 outbound는 anyopen하여 사용할 수 있도록 합니다.
- 7) 과제 종료 전 실행 중인 테스트 및 부하를 중지하여 서버에 문제가 없도록 해야 합니다.
- 8) Tag 정보가 맞지 않거나 권한 문제가 생겨 채점이 불가하지 않도록 주의합니다.
- 9) 모든 리소스의 이름, 태그, 변수는 대소문자를 구분합니다.
- 10) 경기 시작 전 기본 서비스들은 모두 삭제합니다.
- 11) 문제지에서 제공되지 않는 값은 AWS Well-Architected Framework 6 Pillars을 참고하여 적절한 값으로 구성해야 합니다.
- 12) Cloudshell에서 채점스크립트를 실행해 채점을 진행합니다.

No 1. CDN

해당 과제 풀이는 버지니아 북부(us-east-1) 리전에서 진행합니다.

사용자의 디바이스 환경(모바일/데스크탑)을 자동으로 감지하여 최적화된 크기의 이미지를 제공하는 CDN 시스템을 구축합니다. CloudFront의 엣지 컴퓨팅 기능을 활용하여 디바이스별 이미지 리사이징 및 캐싱을 수행합니다.

1. S3

지급받은 배포파일의 원본 이미지를 origin/ 경로에 업로드합니다. S3 Bucket은 퍼블릭 액세스를 완전히 차단하며, CloudFront의 OAC를 통해서만 접근이 가능하도록 구성합니다. 버킷 버전 관리를 활성화합니다.

- S3 Bucket Name : wsc2026-cdn-asset-<선수 비번호>

2. CloudFront

CloudFront 배포를 생성하여 위에서 생성한 S3 버킷을 오리진으로 연결합니다. OAC를 생성하여 S3 오리진에 적용하고, S3 버킷 정책에 CloudFront 서비스 프린시플의 접근을 허용합니다. HTTP 요청은 HTTPS로 리다이렉트되도록 설정합니다. 캐시 정책은 쿼리 스트링을 캐시 키에 포함하도록 구성하여, 동일한 이미지라도 디바이스 타입에 따라 서로 다른 캐시가 유지되도록 합니다. 오리진 요청 정책에도 쿼리 스트링이 오리진으로 전달되도록 설정합니다. 전 세계 모든 CloudFront Edge Location을 사용할 수 있도록 구성합니다.

- CloudFront Distribution Name : wsc2026-cdn

- Cache Policy Name : wsc2026-cache-policy

- Origin Request Policy Name : wsc2026-origin-policy

3. CloudFront Function (Viewer Request)

Viewer Request 단계에 CloudFront Functions를 배포합니다. 해당 함수는 디바이스 타입을 판별하여 모바일 기기로 접속한 경우 쿼리 스트링에 w=480&h=320&type=mobile을, 데스크탑으로 접속한 경우 w=1920&h=1080&type=desktop을 동적으로 삽입합니다. 디바이스 판별 방식은 CloudFront-Is-Mobile-Viewer, CloudFront-Is-Desktop-Viewer 헤더 또는 User-Agent 헤더를 활용하는 등 동등한 방법으로 구현할 수 있으며, 이미 쿼리 스트링이 존재하는 요청에 대해서는 기존 값을 유지하고 변환하지 않습니다.

- CloudFront Function Name : wsc2026-device-detect

- CloudFront Function Runtime : cloudfront-js-2.0

4. CloudFront Function (Viewer Response)

Viewer Response 단계에 별도의 CloudFront Functions를 배포합니다. 해당 함수는 클라이언트에게 응답을 반환하기 전에 X-Device-Type 헤더에 디바이스 타입(mobile 또는 desktop)을, X-Resized 헤더에 true 값을 추가합니다. 디바이스 타입은 쿼리 스트링의 type 파라미터 값을 기반으로 판별합니다.

- CloudFront Function Name : wsc2026-response-header

- CloudFront Function Runtime : cloudfront-js-2.0

5. Lambda@Edge (Origin Response)

Origin Response 단계에 Lambda@Edge 함수를 배포합니다. 해당 함수는 캐시 미스가 발생하여 오리진에서 응답이 돌아올 때 실행됩니다. 쿼리 스트링의 w, h 파라미터를 파싱하여 S3 원본 이미지를 해당 크기로 리사이징한 후 응답 본문을 교체합니다. 리사이징된 이미지는 S3 버킷의 resized/ 경로에 {type}_{원본파일명}_{yyyyMMdd_HHmss}.png 형식으로 저장합니다. yyyyMMdd_HHmss는 KST를 기준으로 합니다. Lambda에서 사용되는 IAM Role은 Administrator 등 과도한 정책을 사용하지 않고 최소한의 정책을 사용해 구성하도록 합니다.

- Lambda Function Name : wsc2026-resize

- Lambda Runtime : python 3.12

- Resize File Name EX : mobile_worldskills_20260424_091500.png

- IAM Role Name : wsc2026-resize-role
- IAM Role Policy : wsc2026-resize-policy

No 2. Keycloak

해당 과제 풀이는 서울(ap-northeast-2) 리전에서 진행합니다.

Keycloak을 자체 IdP로 구축하고, SAML 2.0 페더레이션을 통해 AWS 콘솔에 SSO 로그인하는 시스템을 구성합니다. 그룹에 따라 서로 다른 AWS 권한이 부여되도록 합니다.

1. VPC

VPC와 서브넷을 구성합니다. ALB를 위한 퍼블릭 서브넷은 서로 다른 가용영역에 2개, Keycloak EC2를 위한 프라이빗 서브넷은 1개를 생성합니다. 프라이빗 서브넷의 인스턴스가 외부 패키지를 설치할 수 있도록 적절한 네트워크 구성을 합니다.

- VPC Name : wsc2026-keycloak-vpc
- VPC CIDR : 10.20.0.0/16
- Public Subnet A : 10.20.1.0/24 (wsc2026-public-subnet-a)
- Public Subnet B : 10.20.2.0/24 (wsc2026-public-subnet-b)
- Private Subnet A : 10.20.10.0/24 (wsc2026-private-subnet-a)

2. Security Group

ALB용과 Keycloak EC2용 Security Group을 각각 생성합니다. 외부에서는 ALB를 통해서만 Keycloak에 접근할 수 있어야 하며, SSH나 외부 직접 접근 등을 통해 Keycloak EC2에 접근하는 것은 불가능해야 합니다. 최소 권한 원칙에 따라 필요한 포트만 허용하도록 합니다.

- ALB Security Group Name : wsc2026-keycloak-alb-sg
- Keycloak Security Group Name : wsc2026-keycloak-sg

3. EC2

프라이빗 서브넷에 EC2 인스턴스를 생성하고 Keycloak을 배포합니다. EC2에는 퍼블릭 IP를 부여하지 않으며, SSH를 사용하지 않고 인스턴스에 접근할 수 있는 AWS 관리형 서비스를 활용합니다. 이를 위해 필요한 IAM 권한을 EC2에 부여합니다.

- Instance Name : wsc2026-keycloak
- Instance Type : t3.medium
- AMI : Amazon Linux 2023
- IAM Role Name : wsc2026-keycloak-ec2-role
- Keycloak admin Password : Skill53#!!@#

4. ALB

Keycloak 앞단에 Application Load Balancer를 배치하여 외부에서 Keycloak으로 접근이 가능하도록 80 포트 리스너를 사용하도록 합니다.

- ALB Name : wsc2026-keycloak-alb
- ALB Scheme : Internet-facing
- Target Group Name : wsc2026-keycloak-tg

5. Keycloak Realm 및 사용자 구성

Keycloak 관리 콘솔에 접속하여 AWS SSO를 위한 Realm을 생성합니다. Realm 내에 두 개의 그룹과 사용자를 생성하고, 각 사용자를 해당 그룹에 소속시킵니다.

- Realm Name : wsc2026-aws
- Group : dev-team, infra-team
- User : dev-user (dev-team), infra-user (infra-team)
- dev-user Password : Skills_dev53%\$%
- infra-user Password : Skills_infra53#@#

6. SAML Client 구성

wsc2026-aws Realm에 AWS용 SAML Client를 생성합니다. AWS가 SAML IdP로부터 수신하는 Assertion에는 어떤 Role을 부여할지, 세션을 어떤 이름으로 식별할지, 세션 유지 시간은 얼마인지에 대한 정보가 포함되어야 합니다.

7. AWS IAM 구성

Keycloak Realm에서 SAML 메타데이터를 내보내어 AWS IAM에 SAML Identity Provider로 등록합니다. 이후 두 개의 IAM Role을 생성하며, 각 Role은 등록된 SAML Provider를 통한 페더레이션 로그인을 허용하도록 신뢰 정책을 구성합니다. dev-team으로 로그인한 사용자는 ap-northeast-2 리전의 EC2와 S3의 리소스를 조회만 할 수 있어야 합니다. infra-team으로 로그인한 사용자는 EC2, S3, VPC, IAM의 리소스를 조회할 수 있으며, 추가로 EC2 인스턴스의 시작과 중지도 가능해야 합니다. 하지만 "protected" 태그 인스턴스는 불가능하도록 설정합니다.

- SAML Provider Name : wsc2026-keycloak-idp
- IAM Role (dev) : wsc2026-dev-role
- IAM Policy (dev) : wsc2026-dev-policy
- IAM Role (infra) : wsc2026-infra-role
- IAM Policy (infra) : wsc2026-infra-policy

No 3. Container logging

해당 과제 풀이는 도쿄(ap-northeast-1) 리전에서 진행합니다.

EKS 클러스터에서 실행되는 컨테이너의 로그를 수집하고, 중앙 집중형 로깅 파이프라인을 통해 대시보드에서 조회할 수 있는 환경을 구축합니다. 각 노드에서 Fluent Bit이 로그를 수집하고, OpenTelemetry Collector가 로그를 가공 및 라우팅하며, Loki에 저장된 로그를 Grafana에서 시각화합니다.

1. VPC

EKS 클러스터를 위한 VPC를 구성합니다. 퍼블릭 서브넷 2개와 프라이빗 서브넷 2개를 서로 다른 가용영역에 배치합니다.

- VPC Name : wsc2026-logging-vpc
- VPC CIDR : 10.30.0.0/16
- Public Subnet A : 10.30.1.0/24 (ap-northeast-1a)
- Public Subnet C : 10.30.2.0/24 (ap-northeast-1c)
- Private Subnet A : 10.30.10.0/24 (ap-northeast-1a)
- Private Subnet C : 10.30.20.0/24 (ap-northeast-1c)

2. EKS

EKS 클러스터를 생성하고 관리형 노드 그룹을 구성합니다. 클러스터는 퍼블릭으로 접근 가능하도록 설정하며, 노드 그룹은 프라이빗 서브넷에 배치하도록 합니다.

- Cluster Name : wsc2026-logging-cluster
- Cluster Version : 1.35

- Node Group Name : wsc2026-logging-nodegroup
- Node Instance Type : t3.medium
- Node 수 : 2
- Node AMI : Amazon Linux 2023

3. 애플리케이션 배포

로그를 발생시키기 위한 샘플 애플리케이션을 EKS 클러스터에 배포합니다. 배포파일로 제공되는 deployment.yaml을 활용하여 wsc2026-app 네임스페이스에 배포합니다. 해당 애플리케이션은 HTTP 요청을 통해 원하는 레벨(INFO, WARN, ERROR)의 로그를 JSON 형식으로 출력합니다. 배포된 애플리케이션의 Service를 생성하며 Service 타입을 Loadbalancer로 지정해 엔드포인트를 노출시키도록 합니다.

- API-
- GET /health : 헬스체크
- GET /info : INFO 로그 1건 출력
- GET /warn : WARN 로그 1건 출력
- GET /error : ERROR 로그 1건 출력
- GET /burst?count=N&level=LEVEL : 지정 레벨 로그 N건 출력

로그 출력 형식:

```
{"timestamp":"...", "level":"ERROR", "message":"Failed to connect to database", "service":"log-generator"
}
```

- Namespace : wsc2026-app
- Deployment Name : log-generator
- Port : 8080

4. Fluent Bit

각 워커 노드에서 컨테이너 로그를 수집하는 경량 에이전트를 배포합니다. 클러스터의 모든 노드에서 실행되어야 하며, 이를 위해 적절한 워크로드 타입을 선택합니다. Helm을 활용하여 wsc2026-logging 네임스페이스에 설치합니다. Fluent Bit은 수집한 로그를 Loki로 직접 전송하지 않고, OpenTelemetry Collector의 HTTP 엔드포인트로 전송하도록 Output을 구성합니다. 수집되는 로그에는 Pod 이름, 네임스페이스, 컨테이너 이름 등의 Kubernetes 메타데이터가 포함되어야 합니다. wsc2026-logging 네임스페이스의 로그는 수집 대상에서 제외하여 로깅 시스템 자체의 로그가 순환되지 않도록 합니다.

- Namespace : wsc2026-logging / Helm Release Name : wsc2026-fluent-bit

5. OpenTelemetry Collector

Fluent Bit으로부터 로그를 수신하여 가공한 뒤 Loki와 Prometheus로 전달하는 중앙 수집기를 배포합니다. Helm을 활용하여 wsc2026-logging 네임스페이스에 설치합니다. Collector는 Deployment 형태로 배포하며, 수신에는 Fluent Bit이 전송하는 프로토콜에 맞는 수신기를 구성합니다. 프로세서 단계에서는 로그에 cluster 이름을 리소스 속성으로 추가하고, JSON 형식의 로그 본문을 파싱하여 level, message, service 필드를 로그 속성으로 추출합니다. 파이프라인은 로그와 메트릭 두 가지를 구성합니다. 로그 파이프라인은 Loki의 Push API 엔드포인트로 로그를 내보냅니다. 메트릭 파이프라인은 Count Connector를 활용하여 로그 레벨(INFO, WARN, ERROR) 발생 건수를 메트릭으로 변환하고, Prometheus exporter를 통해 메트릭을 노출합니다. 메트릭 이름은 log_record_count_total로 하며 level 레이블로 구분합니다.

- Namespace : wsc2026-logging / Helm Release Name : wsc2026-otel-collector-opentelemetry-collector

6. Prometheus

OpenTelemetry Collector가 노출하는 로그 기반 메트릭을 수집하는 Prometheus를 배포합니다. Helm을 활용하여 wsc2026-logging 네임스페이스에 설치합니다. OTel Collector의 메트릭 엔드포인트를 scrape target으로 구성하여 log_record_count_total 메트릭을 수집합니다. Grafana에서 데이터 소스로 사용됩니다.

- Namespace : wsc2026-logging / Helm Release Name : wsc2026-prometheus-server

7. Loki

로그 저장소인 Loki를 EKS 클러스터에 배포합니다. Helm을 활용하여 wsc2026-logging 네임스페이스에 설치하며, 단일 바이너리 모드로 구성합니다. OpenTelemetry Collector가 Loki의 Push API로 로그를 전송할 수 있도록 엔드포인트가 클러스터 내부에서 접근 가능해야 합니다.

- Namespace : wsc2026-logging / Helm Release Name : wsc2026-loki

8. Grafana Dashboard

Grafana를 EKS 클러스터에 배포합니다. Helm을 활용하여 wsc2026-logging 네임스페이스에 설치합니다. 외부에서 Grafana 대시보드에 접근할 수 있도록 적절한 Service 타입을 구성합니다. Grafana에 Loki와 Prometheus를 각각 데이터 소스로 등록합니다. wsc2026-app 네임스페이스의 로그를 조회할 수 있는 대시보드를 생성합니다. 대시보드에는 namespace Template Variable을 구성하여 라벨 기반 필터링이 가능하도록 합니다. 대시보드는 Overview 섹션과 Logs by Level 섹션으로 구분하여 구성합니다. Overview 섹션에서는 Prometheus에 저장된 메트릭을 활용하여 로그 레벨별(INFO/WARN/ERROR) 발생 추이를 시계열 그래프로 시각화하며, 각 레벨은 색상으로 구분합니다(INFO=초록, WARN=노랑, ERROR=빨강). Error Rate(%)와 Warn Rate(%)의 추이를 별도 시계열 그래프로 표시하고, 레벨별 분포를 Pie Chart로 시각화합니다. INFO Logs, Error Logs, Warn Logs, 전체 로그 수를 Stat 패널로 표시하고, Error Rate(%)와 Warn Rate(%)를 Gauge 패널로 표시합니다. Logs by Level 섹션에서는 Prometheus 메트릭을 활용하여 레벨별 로그 카운트를 Bar Chart로 시각화합니다.

- Namespace : wsc2026-logging / Helm Release Name : wsc2026-grafana

- Grafana Admin Password : Skill53@@

- Dashboard Name : wsc2026-app-logs

No 4. Workflow

해당 과제 풀이는 싱가포르(ap-southeast-1) 리전에서 진행합니다.

S3에 업로드되는 주문 데이터를 수집하고, Lambda로 검증 및 결제 처리를 수행한 뒤 DynamoDB에 재고 차감과 배송 정보를 기록하는 자동화 파이프라인을 구성합니다.

1. S3

주문 데이터가 업로드되는 버킷을 생성합니다. 버전 관리를 활성화하고, incoming/ 접두사를 생성하여 sample-orders.json 파일을 업로드합니다.

- Bucket Name : wsc2026-order-pipeline

- Versioning : Enabled

- Prefix : incoming/

2. DynamoDB

주문 처리에 필요한 테이블 3개를 생성합니다. Orders 테이블은 처리된 주문 정보를 저장합니다. Inventory 테이블은 상품별 재고 수량을 관리하며, 배포파일로 제공된 inventory-seed.json의 데이터를 사전에 적재합니다. History 테이블은 매 실행의 결과를 기록하며, TTL을 활성화하여 30일 후 자동 만료되도록 합니다. 모든 테이블의 Billing Mode는 On-Demand로 설정합니다.

- Orders Table Name : wsc2026-orders

- Orders Partition Key : order_id (String)

- Orders Sort Key : ordered_at (String)

- Inventory Table Name : wsc2026-inventory

- Inventory partition Key : product_id (String)

- History Table Name : wsc2026-pipeline-history
- History Partition Key : execution_id (String)
- History Sort Key : started_at (String)
- History TTL Attribute : expires_at

3. Lambda

주문 검증 함수와 결제 처리 함수를 각각 생성합니다. 검증 함수는 단일 주문 객체를 입력받아 order_id가 "ORD-" 접두사로 시작할 것, product_id가 빈 문자열이 아닐 것, quantity가 1 이상일 것, unit_price가 0보다 클 것, payment_method가 "CARD" 또는 "BANK_TRANSFER" 중 하나일 것을 검사합니다. 통과 시 {"valid": true, "order": {...}}, 실패 시 {"valid": false, "order": {...}, "errors": [...]}를 반환합니다. 결제 처리 함수는 검증 통과 주문을 입력받아 total_amount를 계산하고(quantity × unit_price), total_usd 필드를 추가하며(환율 고정: 1 USD = 1350 KRW, 소수점 2자리 반올림), payment_status를 "APPROVED"로 설정하고, processed_at을 ISO 8601 형식으로 추가합니다.

- Validator Function Name : wsc2026-order-validator
- Payment Function Name : wsc2026-payment-processor
- Runtime : Python 3.13

4. Step Functions

주문 처리 파이프라인을 오케스트레이션하는 상태 머신을 생성합니다. S3와 DynamoDB는 Lambda를 사용하지 않고 AWS SDK 직접 통합으로 호출합니다. 상태 머신은 {"bucket": "wsc2026-order-pipeline", "key": "incoming/orders.json"} 형태의 입력을 받아 실행됩니다.

- State Machine Name : wsc2026-order-pipeline
- Type : Standard

- Step Functions Logic -

FetchOrders (Task) - S3에서 주문 데이터 파일을 가져옵니다. 응답에서 Body만 추출하여 JSON으로 파싱합니다. ResultPath는 \$.orderData로 설정합니다.

ValidateOrders (Map) - 주문 배열의 각 항목에 대해 wsc2026-order-validator Lambda를 병렬 호출합니다. MaxConcurrency는 5로 설정합니다. Lambda 호출 실패 시 최대 2회 재시도합니다(IntervalSeconds: 2, BackoffRate: 2.0). ResultPath는 \$.validationResults로 설정합니다.

ProcessAndStore (Map) - 검증 결과 배열의 각 항목을 병렬 처리합니다. MaxConcurrency는 10으로 설정합니다. 각 반복 내에서 먼저 Choice State로 valid: true인지 확인하고, true인 경우에만 순차적으로 다음을 수행합니다. valid: false인 항목은 건너뜁니다(Pass State).

- (1) wsc2026-payment-processor Lambda를 호출하여 결제를 처리합니다.
- (2) wsc2026-orders 테이블에 주문 정보를 저장합니다 (SDK 직접 통합).
- (3) wsc2026-inventory 테이블에서 해당 상품의 stock 값을 주문 수량만큼 차감합니다 (SDK 직접 통합, UpdateItem 사용).

DynamoDB 쓰기 실패 시 최대 3회 재시도(IntervalSeconds: 1, BackoffRate: 2.0)하고, 그 외 에러는 Catch하여 건너뜁니다.

RecordResult - 파이프라인 실행 결과를 wsc2026-pipeline-history 테이블에 기록합니다 (SDK 직접 통합). execution_id는 Context Object의 Execution Id, started_at은 Execution StartTime을 활용합니다. status "COMPLETED", total_orders, valid_orders, processed_orders 카운트와 expires_at(현재 + 30일 Unix timestamp)을 포함합니다.

PipelineFailed - FetchOrders 또는 ValidateOrders에서 복구 불가능한 에러 발생 시 전이됩니다. wsc2026-pipeline-history 테이블에 status "FAILED", error, cause를 기록합니다.